



Guide d'Installation et de Compilation Multiplateforme - AirBar (Application Flutter)

Version: 1.0 **Date:** 31 juillet 2026 **Périmètre:** Ce guide couvre **uniquement l'application cliente Flutter** ([airbar/](#)) : installation de l'environnement, récupération du code, configuration et compilation pour Windows, macOS, Linux, Android, iOS et Web.

⚠ Ce guide ne couvre PAS l'installation du backend Serverpod (PostgreSQL, Redis, serveur API). Le backend est supposé déjà installé et accessible (voir la documentation dédiée, ex. [guide-installation-windows-production.md](#) pour la partie serveur). Ici, on ne configure que l'**hôte/port** auquel l'app cliente doit se connecter.



Table des Matières

1. Prérequis Généraux
 2. Récupération du Projet
 3. Configuration des Dépendances (pubspec)
 4. Configuration du Serveur Backend (côté app)
 5. Vérification de l'Environnement Flutter
 6. Compilation par Plateforme
 - 6.1 Windows (Desktop)
 - 6.2 macOS (Desktop)
 - 6.3 Linux (Desktop)
 - 6.4 Android
 - 6.5 iOS
 - 6.6 Web
 7. Problèmes Fréquents de Compilation
 8. Checklist Finale
-



1. Prérequis Généraux

Quel que soit le système d'exploitation utilisé pour compiler, il faut :

Outil	Version minimale	Vérification
Flutter SDK	3.x (Dart SDK ^3.11.0, voir pubspec.yaml)	<code>flutter --version</code>
Git	2.x	<code>git --version</code>
Un IDE	VS Code (+ extensions Flutter/Dart) ou Android Studio	-

Installer Flutter selon l'OS hôte :

- **Windows** : <https://docs.flutter.dev/get-started/install/windows>
- **macOS** : <https://docs.flutter.dev/get-started/install/macos>
- **Linux** : <https://docs.flutter.dev/get-started/install/linux>

💡 **Rappel important** : on ne peut compiler pour **iOS et macOS** qu'à partir d'un **Mac** (Xcode requis). Windows/Linux/Android/Web peuvent être compilés depuis Windows, macOS ou Linux (avec les toolchains adaptées).



2. Récupération du Projet

L'application cliente est un repository **indépendant** du backend :

```
git clone https://github.com/rorophil/airbar.git
cd airbar
```

Contrairement au backend, **aucun outil Docker n'est nécessaire** pour l'app cliente : c'est un projet Flutter standard.



3. Configuration des Dépendances (pubspec)

Ce projet utilise **deux fichiers pubspec templates** au lieu de modifier `pubspec.yaml` directement, afin de basculer facilement entre un backend en dépendance **locale** (chemin relatif, pratique en dev sur la même machine que le serveur) et un backend en dépendance **Git** (pratique pour compiler sur une autre machine, ex. Windows testant contre un Mac, ou build de production) :

- `pubspec_mac.yaml` → `airbar_backend_client` en **path local** (`../airbar_backend/airbar_backend_client`)
- `pubspec_windows.yaml` → `airbar_backend_client` en **dépendance Git** (https://github.com/rorophil/airbar_backend.git)

Choisir la configuration adaptée à votre machine de build

Si le repository `airbar_backend` est cloné à côté (`../airbar_backend`) :

```
# Windows
.\switch_to_git.ps1  # si vous n'avez PAS le backend en local -> dépendance Git
# ou
.\switch_to_local.ps1 # si vous AVEZ ../airbar_backend en local
```

```
# macOS / Linux
./switch_to_local.sh  # si ../airbar_backend est présent en local
./switch_to_git.sh    # sinon, dépendance Git
```

Le nommage des scripts (`_mac/_windows`) reflète l'usage historique du projet (Mac = dev local, Windows = test contre Git), mais **le choix dépend uniquement de la présence ou non du dossier `../airbar_backend` sur la machine de compilation**, pas de l'OS lui-même.

Chaque script :

1. Copie le template choisi vers `pubspec.yaml`
2. Exécute `flutter pub get`



Règle importante

Ne jamais modifier `pubspec.yaml` directement (il est régénéré par les scripts). Pour ajouter/modifier une dépendance, éditer **les deux templates** (`pubspec_mac.yaml` ET `pubspec_windows.yaml`), voir [DEPENDENCIES.md](#) et le script `add_dependency.sh`.

Si aucun script n'est disponible pour votre OS de build ou en cas de doute :

```
flutter pub get
```

après avoir vérifié manuellement que `pubspec.yaml` pointe vers la bonne source pour `airbar_backend_client`.

4. Configuration du Serveur Backend (côté app)

L'app ne contient **aucune URL de serveur en dur**. Au premier lancement (ou via le bouton "**Configuration serveur**" sur l'écran de login), renseigner :

- **Hôte** : IP ou nom d'hôte du serveur Serverpod (ex. `localhost`, `10.211.55.2`, ou l'IP/domaine de production)
- **Port** : `8080` par défaut

La configuration est stockée localement (SharedPreferences) sur l'appareil, elle n'est **pas** versionnée dans Git et n'a aucun impact sur la compilation.

5. Vérification de l'Environnement Flutter

Avant de compiler pour une plateforme donnée, toujours vérifier :

```
flutter doctor -v
```

Activer les plateformes desktop si nécessaire (désactivées par défaut sur certaines installations) :

```
flutter config --enable-windows-desktop  
flutter config --enable-macos-desktop  
flutter config --enable-linux-desktop
```

Lister les appareils/cibles disponibles :

```
flutter devices
```



6. Compilation par Plateforme

Dans toutes les sections ci-dessous, se placer à la racine du projet `airbar/` et avoir exécuté `flutter pub get` (ou un script `switch_to_*`) au préalable.

6.1 Windows (Desktop)

Prérequis (sur la machine Windows) :

- Visual Studio 2022 avec le workload "**Développement Desktop en C++**" (Desktop development with C++)
- Windows 10 SDK (installé avec le workload ci-dessus)

Développement :

```
flutter run -d windows
```

Build de production :

```
flutter build windows --release
```

Artefact généré :

```
build\windows\x64\runner\Release\airbar.exe
```

(et les DLL associées dans le même dossier — à distribuer ensemble, ex. via un zip ou un installateur MSIX/Inno Setup).

Test rapide de l'exécutable :

```
.\build\windows\x64\runner\Release\airbar.exe
```

6.2 macOS (Desktop)

Prérequis (sur un Mac) :

- Xcode (dernière version stable) + Command Line Tools : `xcode-select --install`
- CocoaPods : `sudo gem install cocoapods`

Développement :

```
flutter run -d macos
```

Build de production :

```
flutter build macos --release
```

Artefact généré :

```
build/macos/Build/Products/Release/airbar.app
```

Pour une distribution en dehors du Mac de développement (signature, notariisation Apple), une identité de développeur Apple et une configuration de signing dans Xcode (`macos/Runner.xcworkspace`) sont nécessaires. Non requis pour un usage interne/test.

6.3 Linux (Desktop)

Prérequis (sur une distribution Linux, ex. Ubuntu/Debian) :

```
sudo apt-get update
sudo apt-get install -y clang cmake ninja-build pkg-config libgtk-3-dev liblzma-dev
```

Développement :

```
flutter run -d linux
```

Build de production :

```
flutter build linux --release
```

Artefact généré :

```
build/linux/x64/release/bundle/
```

(dossier complet à distribuer tel quel — contient l'exécutable **airbar** et ses librairies).

6.4 Android

Prérequis :

- Android Studio (fournit le SDK Android et les licences)
- Accepter les licences SDK : **flutter doctor --android-licenses**
- Un appareil Android (mode débogage USB activé) ou un émulateur, pour le test

Développement :

```
flutter run -d <device_id>
```

Build de production (APK, pour installation directe) :

```
flutter build apk --release
```

Artefact : **build/app/outputs/flutter-apk/app-release.apk**

Build de production (App Bundle, pour le Play Store) :

```
flutter build appbundle --release
```

Artefact : `build/app/outputs/bundle/release/app-release.aab`

⚠ Avant une vraie mise en production :

- `applicationId` est actuellement `com.example.airbar` dans `android/app/build.gradle.kts` — à personnaliser (ex. `com.aeroclub.airbar`) avant publication.
- Le build `release` signe actuellement avec la **clé de debug** (`signingConfigs.getByName("debug")`) : suffisant pour un APK de test interne, mais **une vraie configuration de signature (keystore) est indispensable** pour publier sur le Play Store.

6.5 iOS

Prérequis (sur un Mac uniquement) :

- Xcode + Command Line Tools
- CocoaPods
- Un compte développeur Apple (gratuit pour tester sur un appareil personnel, payant pour publication sur l'App Store)

Développement (simulateur ou appareil) :

```
flutter run -d <device_id>
```

Build de production (sans signature, build brut) :

```
flutter build ios --release
```

Build IPA (pour distribution / TestFlight / App Store) :

```
flutter build ipa --release
```


Artefact : `build/ios/ipa/*.ipa`

La signature (certificats, profils de provisionnement) se configure dans Xcode via `ios/Runner.xcworkspace` (onglet **Signing & Capabilities**) avant de pouvoir générer un IPA distribuable.

6.6 Web

Prérequis : aucun outil supplémentaire (Chrome recommandé pour le dev).

Développement :

```
flutter run -d chrome
```

Build de production :

```
flutter build web --release
```

Artefact généré :

```
build/web/
```

Déploiement : copier le contenu de `build/web/` sur un serveur HTTP statique (nginx, Apache, etc.) configuré pour servir `index.html`. Si l'app est servie depuis un sous-dossier (pas la racine du domaine), ajuster le `--base-href` :

```
flutter build web --release --base-href "/airbar/"
```

7. Problèmes Fréquents de Compilation

"Cannot find airbar_backend_client" / erreurs de résolution de package

Cause : `pubspec.yaml` pointe vers un chemin local (`../airbar_backend/...`) qui n'existe pas sur la machine de build.

Solution : utiliser la configuration Git (`switch_to_git.ps1/.sh`) ou cloner `airbar_backend` au bon endroit relatif, puis `flutter pub get`.

"Bad Request" / incompatibilité client-serveur après compilation

Cause : le client Serverpod compilé ne correspond pas à la version du serveur en face.

Solution : vérifier que le backend a bien été régénéré (`serverpod generate`) et que la dépendance `airbar_backend_client` utilisée pointe vers la bonne version/branche, puis `flutter clean && flutter pub get`.

Windows : `flutter build windows` échoue avec une erreur CMake/MSVC

Cause : workload Visual Studio "Développement Desktop en C++" manquant.

Solution : ouvrir le **Visual Studio Installer** et ajouter ce workload, puis relancer `flutter doctor`.

macOS/iOS : erreurs CocoaPods (`pod install` échoue)

Solution :

```
cd ios    # ou macos
pod repo update
pod install
cd ..
```

```
flutter clean  
flutter pub get
```

Linux : `flutter build linux` échoue avec des libs GTK manquantes

Solution : installer les paquets listés en 6.3 (`libgtk-3-dev` notamment).

Android : échec de build lié à Java/Gradle

Cause : version de JDK incompatible (le projet utilise `JavaVersion.VERSION_17`, voir [android/app/build.gradle.kts](#)).

Solution : utiliser un JDK 17 (celui fourni avec Android Studio récent convient) et vérifier `flutter doctor -v` pour la ligne Java/Android toolchain.

Build générique qui échoue après un `git pull`

```
flutter clean  
flutter pub get
```

puis relancer la commande de build.



8. Checklist Finale

- ☐ `flutter doctor -v` sans erreur bloquante pour la plateforme ciblée
- ☐ Bon fichier pubspec appliqué (`switch_to_local/switch_to_git`) selon la disponibilité de `../airbar_backend`
- ☐ `flutter pub get` exécuté sans erreur
- ☐ `flutter run -d <plateforme>` fonctionne en développement
- ☐ `flutter build <plateforme> --release` génère l'artefact attendu
- ☐ Configuration serveur (hôte/port) testée dans l'app une fois installée
- ☐ (Android) `applicationId` et signature vérifiés avant publication

- ☐ (iOS/macOS) Signing & Capabilities configurés avant distribution
-

Documents Liés

- [QUICK_START.md](#) — configuration rapide du poste de dev (Mac/Windows)
- [WORKFLOW_DEV.md](#) — workflow Git multi-machines
- [DEPENDENCIES.md](#) — gestion des dépendances (pubspec templates)
- [guide-installation-windows-production.md](#) — installation du **backend** (PostgreSQL, Redis, Serverpod) sur Windows en production